

RMFS Simulator Documentation

Contents

1	Assumptions	2
2	General Description	2
3	Code Structure	2
4	Input Parameters	3
4.1	Order Generation	3
5	Domain Entities	4
5.1	Order	4
5.2	Pod	4
5.3	Robot	4
5.4	Workstation	4
5.5	Task and Visit	5
5.6	Enumerations	5
5.7	Warehouse	5
5.7.1	Warehouse Layout	5
6	Simulator Engine	7
6.1	SimulatorConfig - immutable parameters	7
6.2	SimulatorState - mutable per-run state	7
6.3	Simulator - orchestrator	7
6.4	Event Handlers	8
7	KPIs Collection	11
8	Optimization Approaches	12
8.1	Greedy Policies	12
8.1.1	Assignment Order-Workstation	12
8.1.2	Task Design and Sequencing	12
8.2	Global optimization model	12
8.2.1	Time-space network	13
8.2.2	First Stage: Order-Workstation and Order-Item-Pod Assignments	13
8.2.3	Second Stage: Task Design and Scheduling	13
8.2.4	Local Search Stage 1	14
8.2.5	Local Search Stage 2	15

1 Assumptions

For the sake of simplicity, the simulator is based on the following simplifying assumptions.

- Replenishment missions are not considered, therefore no inventory constraints are modelled. Each pod contains a limited set of SKUs, but the exact quantity or volume of each item is not known.
- Technical aspects of the robots, such as energy management and low-level routing, are outside the scope of this simulator and are not modelled.
- At the end of a mission, the pod will return to its storage position.

2 General Description

The discrete-event simulator emulates the operational dynamics of an Amazon-style RMFS (Robotic Mobile Fulfillment System), based on a *parts-to-picker* approach: robots bring pods to workstations, where operators pick items and process orders.

The simulator supports two decision-making modes:

1. **Cyclic optimisation:** every Δt_{opt} a mathematical model is solved for task generation and sequencing.
2. **Greedy policies:** online decisions are made based on policies with limited lookahead.

In both modes, the assignment of a task to a robot is always managed via a greedy policy based on the minimization of the distance between the resting position of the robot and the storage location of the pod used by the task.

3 Code Structure

The project is organised on four logical levels: **core** (shared entities and data structures), **sim** (DES engine and event handlers), **opt** (assignment policies) and **stat** (KPIs collection management).

Path	Role
run_simulation.py	Entry point: configures logging, builds the warehouse, starts the simulation
experiments.csv	Centralised numerical parameters
scripts/core/enums.py	Enumerations for discrete states and event types
scripts/core/entities.py	Dataclasses for Order , Pod , Robot , Workstation and Task .
scripts/core/queues.py	Generic priority queue (min-heap with O(1) lookup)
scripts/core/warehouse.py	Physical layout and generation, distance utilities
scripts/sim/Simulator.py	Main DES engine: clock, event loop, dispatch table
scripts/sim/event_handler.py	Handler function for each event type
scripts/stat/StatManager.py	Main collector of KPIs
scripts/stat/core.py	Dataclasses to support the collection of KPIs
scripts/opt/policies.py	Greedy assignment policies
scripts/opt/OptManager.py	The class OptManager controls the optimisation process: precomputes the time-space network and converts binary variables in Task objects
scripts/opt/local_search_stage1.py	Definition of the local search algorithm used by the optimization solver for the stage 1
scripts/opt/local_search_stage2.py	Definition of the local search algorithm used by the optimization solver for the stage 2
scripts/opt/stage2_data.py	Dataclass for the second stage of the optimization process
scripts/opt/utils.py	Here is defined a function that converts the binary variables into Task used by the simulator.

Table 1: Folder and file structure of the project.

4 Input Parameters

The following table lists the simulator configuration parameters. The parameters that define order generation and pod processing times at workstations have a domain defined by [BJS24]¹.

The numerical values present are those adopted by [BJS24] or fixed based on the literature; parameters marked with ??? are yet to be defined.

Parameter	Description
<i>Layout and capacity</i>	
NUM_PODS	Total number of pods ($\text{GRID_ROWS} \times \text{GRID_COLS}$)
NUM_SKUS	Number of distinct SKUs
NUM_SKUS_PER_POD	Target number of distinct SKUs for each pod
NUM_ROBOTS	Number of autonomous mobile robots
NUM_WORKSTATIONS	Number of picking stations
GRID_ROWS / GRID_COLS	Pod grid dimensions
WS_ORDER_CAPACITY	Maximum number of open orders per station
WS_RELEASED_TASK_CAPACITY	Number of tasks released per workstation
<i>Operational times</i>	
ROBOT_SPEED	Robot speed (cells/min, $\approx 0,5$ m/s)
POD_PROCESS_TIME	Processing time of a pod at the workstation
ITEM_PROCESS_TIME	Picking time of an item at the workstation
<i>Order generation</i>	
INTERARRIVAL_TIME_ORDER	Mean order inter-arrival time (min)
PROB_1_ITEM_ORDER	Probability that an order contains a single item
GEO_DIST_PARAM_ORDER	Geometric parameter for multi-item orders
<i>Simulation control</i>	
TIME_HORIZON	Total simulation duration (min)
DELTA_T_OPT	Optimiser call interval

Table 3: Numerical parameters of the simulator divided by functionality.

The order generation algorithm is reported here for completeness, as it operationalizes the stochastic rules defined by the parameters table.

4.1 Order Generation

Order generation follows the model of Barnhart et al [BJS24]:

Algorithm 1 Order Generation

```

1: Sample  $u \sim \text{Uniform}(0, 1)$ 
2: if  $u < \text{PROB\_1\_ITEM\_ORDER}$  then
3:    $\text{order\_size} \leftarrow 1$ 
4: else
5:   Sample  $g \sim \text{Geometric}(p)$  with  $p = \text{GEO\_DIST\_PARAM\_ORDER}$  until  $g \leq 15$ 
6:    $\text{order\_size} \leftarrow g + 1$ 
7: end if
8: for  $i = 0$  to  $\text{order\_size} - 1$  do
9:   Sample  $\text{SKU}_i$  from a truncated normal distribution centered in  $N/2$  and with standard deviation  $N/6$ 
    $\triangleright N$  is the total number of unique SKUs in the warehouse
10: end for
```

¹GitHub repository <https://github.com/alexschmid3/warehouse-task-assignment>.

5 Domain Entities

Entities are implemented as Python dataclasses and represent the state of the system during the simulation. The main ones are: `Order`, `Pod`, `Robot`, `Workstation`, `Task` and `Visit`.

5.1 Order

Tracks a customer order from arrival to completion.

Field	Type	Description
<code>order_id</code>	<code>int</code>	Unique identifier
<code>arrival_time</code>	<code>float</code>	Order arrival time
<code>order_size</code>	<code>int</code>	Number of items required
<code>items_required</code>	<code>list[int]</code>	Full list of required items
<code>items_pending</code>	<code>list[int]</code>	Items not yet picked (decremented as items are picked)
<code>workstation_id</code>	<code>int None</code>	Assigned workstation (<code>None</code> if not yet assigned)
<code>status</code>	<code>OrderStatus</code>	Lifecycle state: <code>BACKLOG</code> → <code>WAITING</code> → <code>OPEN</code> → <code>CLOSED</code>

5.2 Pod

Represents a storage pod. When idle, the pod is at its `storage_location`; when on a mission, its status becomes `BUSY`, and the location is not known.

Field	Type	Description
<code>pod_id</code>	<code>int</code>	Unique identifier
<code>storage_location</code>	<code>int</code>	Identifier of the storage cell
<code>items</code>	<code>list[int]</code>	SKUs contained in the pod
<code>status</code>	<code>PodStatus</code>	<code>IDLE</code> or <code>BUSY</code>

5.3 Robot

Each robot is always in one of two states: `IDLE` (available) or `BUSY` (executing a task).

Field	Type	Description
<code>robot_id</code>	<code>int</code>	Unique identifier
<code>position</code>	<code>id</code>	Current position in the grid in terms of cell id (the information is known only when the robot is not moving)
<code>status</code>	<code>RobotStatus</code>	<code>IDLE</code> or <code>BUSY</code>

5.4 Workstation

The workstation manages two separate buffers: one for pods (`picking_buffer`) and one for orders waiting to be opened (`order_buffer`). Note that the `status` refers to the picking operation, not to whether any orders are currently open at the workstation.

Field	Type	Description
workstation_id	int	Unique identifier
order_capacity	int	Maximum simultaneously open orders
released_task_capacity	int	Maximum released tasks per workstation
position	int	Cell identifier of the position
pod_process_time	float	Pod processing time at a workstation
item_process_time	float	Time for a picking operation at a workstation
opened_orders	set[int]	Orders currently being processed
order_buffer	list[int]	Orders waiting to be opened
picking_buffer	list[int]	Pods waiting (excluding the active one) for a picking operation
active_tasks	set[int]	IDs of released tasks allocated to a robot
released_tasks	set[int]	IDs of released tasks not yet allocated
status	WorkstationPickingStatus	IDLE or BUSY

5.5 Task and Visit

A **Task** is a mission assigned to a pod-robot pair, composed of an ordered sequence of stops (**stops**). Each stop is a **Visit**, which specifies the destination workstation, the orders served (**orders**), and the items to be picked (**items**). The **priority** field of the task determines the dispatching order in the queue.

5.6 Enumerations

OrderStatus, **RobotStatus**, **PodStatus** and **WorkstationPickingStatus** define the discrete states of the entities, making state transitions explicit in the simulation logic.

5.7 Warehouse

The **Warehouse** class collects all information and state variables of the system entities. In particular, its attributes include the lists of pods, workstations, and robots.

This class also provides methods to generate the warehouse layout based on the number of pods along the x -axis, the number of pods along the y -axis, the number of workstations and the number of robots.

5.7.1 Warehouse Layout

The warehouse layout is generated from **grid_rows** and **grid_cols**. Pods are arranged in a rectangular grid; after every two columns of pods, an aisle of width two cells is inserted to allow robot movement. The resulting physical dimensions are:

$$X = \text{grid_rows} + 2 \left\lfloor \frac{\text{grid_rows} - 1}{2} \right\rfloor + 2 \cdot \text{MARGIN} - 1, \quad Y = \text{grid_cols} + 2 \cdot \text{MARGIN} - 1$$

Workstations are placed along the perimeter, preferably on the bottom side in a symmetric configuration. Robots are initialized at random, non-overlapping positions within the warehouse. The travel time between two cells is computed as the sum of two components: a nominal travel time, obtained by dividing the Manhattan distance by **robot_speed**, and a positive stochastic component accounting for potential delays due to congestion or other effects.

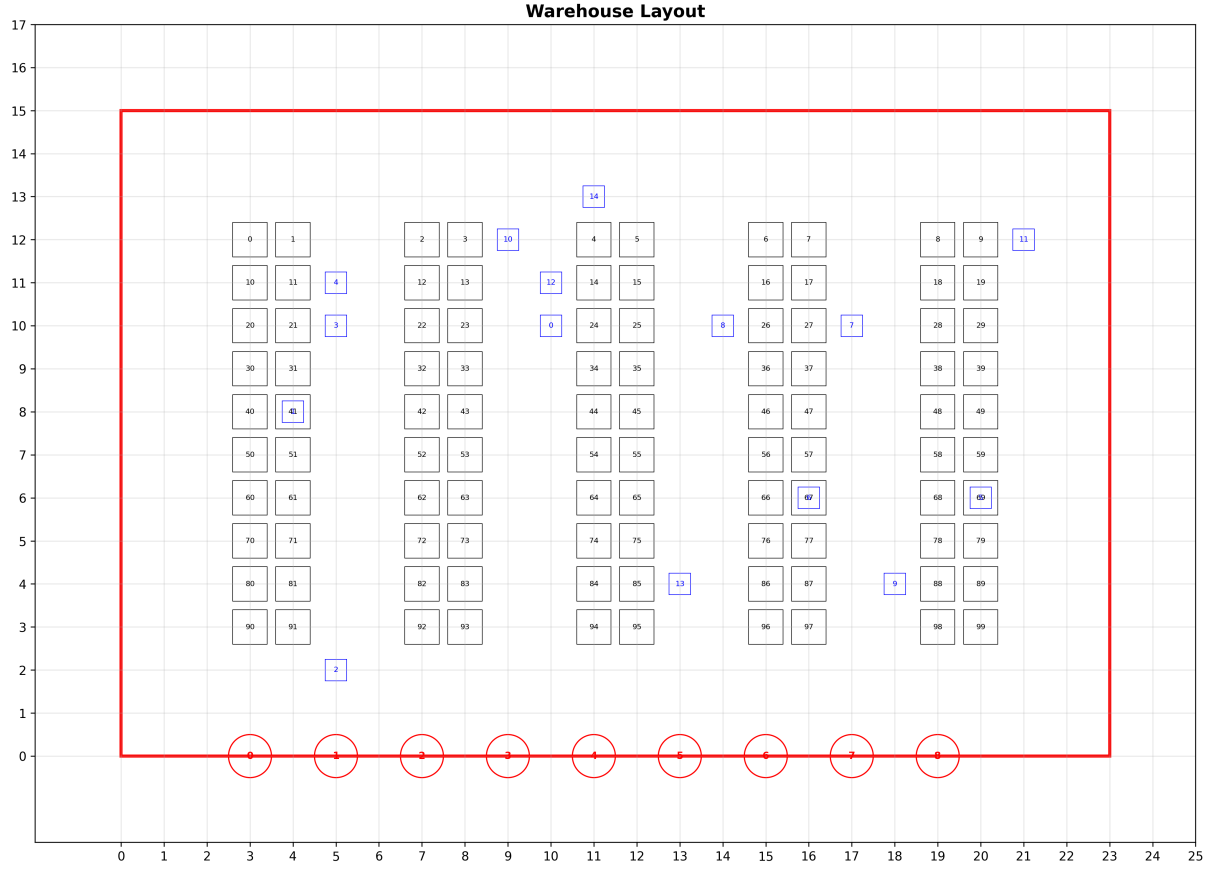


Figure 1: Example layout created with `GRID_ROWS = 10`, `GRID_COLS = 10`, `NUM_ROBOTS = 15` and `NUM_WORKSTATION = 9`. Black cells are the `storage_location` of each pod, red ones belong to workstations, and blue ones are the initial positions of the robots.

6 Simulator Engine

The simulation engine is split across three Python objects to cleanly separate immutable configuration from mutable per-run state.

6.1 SimulatorConfig - immutable parameters

A dataclass that holds every parameter that does not change across replicas. It is constructed once and shared by all `run()` calls on the same `Simulator` instance.

Attribute	Type	Description
<code>order_gen_config</code>	<code>list[float]</code>	<code>[interarrival_time, prob_1_item, geo_param]</code> , parameters for order arrival generation
<code>time_horizone</code>	<code>float</code>	Time horizon of the simulator in seconds..
<code>warm_up</code>	<code>float</code>	Seconds discarded from statistics at run start. Default 0.
<code>optimization_enabled</code>	<code>bool</code>	<code>False</code> → greedy policies; <code>True</code> → optimiser
<code>optimization_interval</code>	<code>float</code>	Time interval between two optimization in seconds. Default 15*60

6.2 SimulatorState - mutable per-run state

A plain dataclass holding all quantities that change during a simulation run. A fresh `SimulatorState` is constructed at the beginning of every `run()` call, guaranteeing that consecutive calls on the same `Simulator` object produce fully independent replicas with no shared mutable state.

Attribute	Type	Description
<code>current_time</code>	<code>float</code>	Current simulation time (seconds)
<code>warehouse</code>	<code>Warehouse</code>	Physical environment and live state of all entities
<code>future_events</code>	<code>PriorityQueue[Event]</code>	Future events ordered by timestamp; <code>RELEASE_TASK</code> breaks ties first
<code>orders_in_system</code>	<code>PriorityQueue[Order]</code>	Received orders ordered by arrival time; <code>BACKLOG</code> orders have priority
<code>released_tasks</code>	<code>PriorityQueue[Task]</code>	Tasks ready for robot assignment
<code>active_tasks</code>	<code>dict[int, Task]</code>	Executing tasks, keyed by <code>task_id</code>
<code>orders_counter</code>	<code>int</code>	Monotonic counter for order IDs
<code>task_counter</code>	<code>int</code>	Monotonic counter for task IDs
<code>event_count</code>	<code>int</code>	Total events processed (for logging)
<code>closed_orders</code>	<code>int</code>	Completed orders (for logging)

6.3 Simulator - orchestrator

The `Simulator` class owns the immutable configuration, the random generator and the `StatManager`. Its constructor accepts a `warehouse_factory`, a zero-argument callable that returns a fresh `Warehouse`, instead of a `Warehouse` instance directly. This design choice is intentional: calling `run()` multiple times in the main script produces independent replicas automatically, because each call rebuilds the full `SimulatorState` (including a new `Warehouse`) from scratch before processing any event.

Attribute	Type	Description
<code>RANDOM_GENERATOR</code>	<code>numpy.Generator</code>	Random generator for reproducibility
<code>config</code>	<code>SimulatorConfig</code>	Immutable run parameters
<code>state</code>	<code>SimulatorState</code>	Mutable state of the current run (rebuilt on every <code>run()</code> call)
<code>STAT_MANAGER</code>	<code>StatManager</code>	KPI collector (also rebuilt on every <code>run()</code> call)
<code>_warehouse_factory</code>	<code>Callable[[], Warehouse]</code>	Factory invoked once per <code>run()</code> to produce a pristine warehouse
<code>OPT_MANAGER</code>	<code>OptManager</code>	Optimization manager (also rebuilt on every <code>run()</code> call), which precomputes the time-space network and provides the interface to the optimizer.

The main event loop inside `run()` follows the classical DES structure:

1. Rebuild `state` and `STAT_MANAGER` from a fresh warehouse.
2. Push the first `ARRIVAL_ORDER` event at $t \approx 0$.
3. If optimization is enabled, push the first `RUN_OPTIMIZER` event at $t \approx 1min$.
4. Pop the event with the smallest timestamp from `state.future_events`.
5. Advance `state.current_time` to the event time.
6. Dispatch to the corresponding handler via the dispatch table.
7. Repeat until the queue is empty or `TIME_HORIZON` is reached.
8. Write and reset statistics via `STAT_MANAGER`.

To be precise, the first `ARRIVAL_ORDER` event will make arrive 30 orders simultaneously to mimic a initial situation of a already working warehouse.

6.4 Event Handlers

Each `EventType` is associated with a handler function with signature `handler(event, state, sim)`, where `state` is the `SimulatorState` holding all mutable queues and counters, and `sim` provides access to the immutable configuration (`sim.config`) and the random generator (`sim.RANDOM_GENERATOR`). The table summarises the current implementation status.

Note that once a task is extracted from the priority queue in `START_TASK`, it is then passed along between events as input and moves through the simulation.

In the `Simulator` class, when the `future_events` queue is initialized, ties between simultaneous events are resolved by prioritizing `RELEASE_TASK` events. This is because the greedy policy used for task generation must consider the full set of released tasks in order to avoid producing redundant tasks, which could lead to inconsistencies in the simulation. For the same reason, at the end of the `END_PICKING` event, if a new order is opened at the workstation, no new tasks are designed or released, as this will be handled during the `OPEN_ORDER` process. The reason behind these design choices will become clearer when the greedy policies are introduced.

The following flowcharts illustrate the dependencies between events and the overall system dynamics.

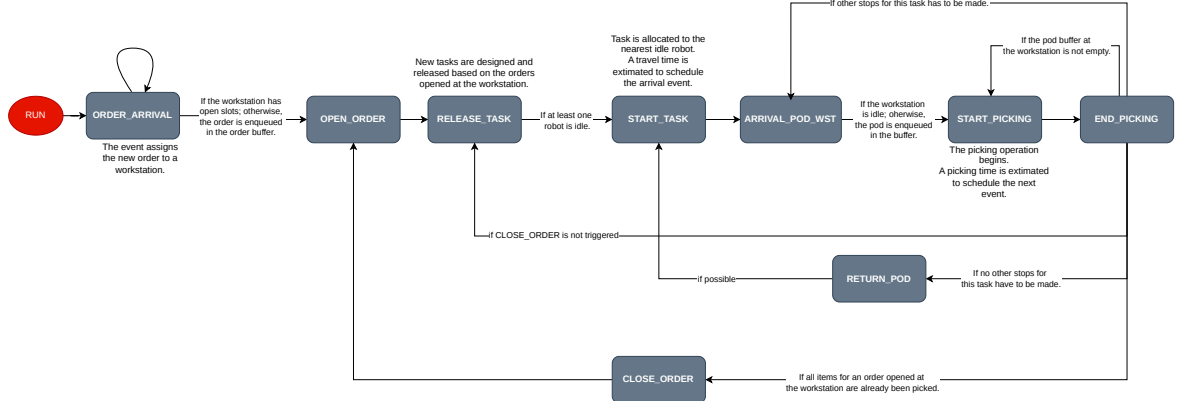


Figure 2: Graph showing the relationship between events when `optimization_enabled = False`. Arrows indicate which event will be scheduled next.

Event	Event.info	Responsibility
ARRIVAL_ORDER	None	Generates a new order, store it in <code>state.orders_in_system</code> and schedules the next order arrival; without the optimiser, assigns it directly to a workstation.
OPEN_ORDER	Order	Sets the order to <code>OPEN</code> , inserts it into <code>opened_orders</code> and, if possible, generates the associated tasks and triggers <code>RELEASE_TASK</code> .
RELEASE_TASK	Task	Releases tasks, if optimizer is enabled all <code>RELEASE_TASK</code> events will be scheduled at the end of the optimization; otherwise events will be scheduled when tasks are designed.
START_TASK	None	Verify that among the released tasks there is at least one that can be started (i.e., whose corresponding pod is idle). If so, assign an idle robot (if available) to it and set both the robot and the pod to <code>BUSY</code> .
ARRIVAL_POD_WST	Task	Handles pod arrival at the workstation: inserts it into the <code>picking_buffer</code> and triggers <code>START_PICKING</code> if the workstation is idle.
START_PICKING	Task	Starts picking, sets the workstation to <code>BUSY</code> and schedules <code>END_PICKING</code> according to the picking process times.
END_PICKING	Task	Completes the picking operation by updating the <code>items_pending</code> field of the involved orders. If necessary, it triggers <code>CLOSE_ORDER</code> , and schedules new pod movements or their return to storage based on the remaining stops in the task. Since the number of released tasks for this workstation has been decremented, new tasks can now be created and released.
CLOSE_ORDER	Order	Closes the order when completed, frees a slot at the workstation and trigger <code>OPEN_ORDER</code> if possible.
RETURN_POD	Task	Returns the pod to its <code>storage_location</code> , sets robot and pod to <code>IDLE</code> and enables new tasks to start.
RUN_OPTIMIZER	None	Invokes the optimiser to assign backlog orders and generate tasks. To maintain coherence between simulation and optimization, active tasks are considered as completed while released tasks not yet allocated are overwritten.

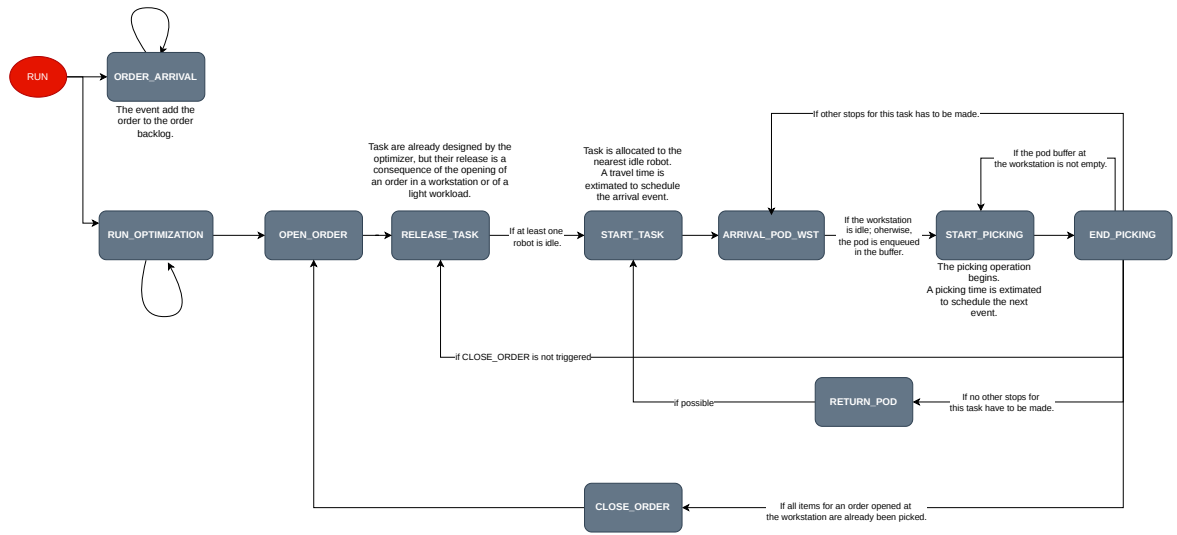


Figure 3: Graph showing the relationship between events when `optimization_enabled = True`. Arrows indicate which event will be scheduled next.

7 KPIs Collection

The main objective of this simulator is to evaluate two different optimization approaches in a controlled and reproducible environment; hence, several relevant statistics must be collected.

The collected statistics are grouped into three categories:

- **ResourceTracker**: this type of statistic measures the state frequency of a given entity over time. It is used, for example, for both robots and workstations, and captures respectively the robot utilization rate and the workstation picking time ratio.
- **OrderFlowTracker**: this statistic captures the average order flow time, grouped by order size, from the `BACKLOG` state to the `CLOSED` state.
- **TimeWeightedMeanTracker**: this type of statistic is used to compute time-averaged KPIs, such as the average number of open orders per workstation and the average number of pods simultaneously out of their storage location.
- **Counter**: this type of statistic is used to count occurrences of something; in particular, it is used to track the throughput of the system.

Finally, all statistics can be aggregated to compute global performance indicators and support system-level analysis.

8 Optimization Approaches

8.1 Greedy Policies

8.1.1 Assignment Order-Workstation

The assignment order-workstation is performed whenever an order arrives in the system. The greedy policy implemented is the following²:

Algorithm 2 Assignment order-workstation

- 1: Select `workstation_id` as the open workstation minimizing the total number of orders, computed as the sum of queued and currently open orders
-

In this policy, the sequencing of orders at a given workstation is neglected, as their priority is solely given by their arrival time.

8.1.2 Task Design and Sequencing

Task design is performed greedily, considering only the orders currently open at a given workstation, with the objective of maximizing pile-on that is, minimizing the number of pods required to complete all open orders at the workstation. During the algorithm, all released tasks associated with the given workstation that have not yet been allocated are redesigned to account for newly required items. This is why the realising of already designed tasks must be prioritized in order to avoid inconsistencies.

In practice, task sequencing is neglected, as priority is based exclusively on the time at which tasks are designed and released.

Algorithm 3 Task Design

- 1: Compute `uncovered_skus` as the set of SKUs required by open orders at the workstation that are not yet covered by active tasks
 - 2: **if** `uncovered_skus` is empty **then**
 - 3: **return**
 - 4: **end if**
 - 5: **while** the number of designed tasks is below the maximum number of tasks that can be generated **and** `uncovered_skus` is not empty **do**
 - 6: Rank pods according to the number of items in `uncovered_skus` they can serve (breaking ties by distance)
 - 7: Select the highest-ranked pod
 - 8: Define a `Visit` based on the subset of orders that benefit from the selected pod
 - 9: Create a task containing the defined `Visit`
 - 10: **end while**
 - 11: **return** Tasks designed
-

Some observations have to be made. First, this policy does not allow tasks with multiple visits to be defined, as it does not have global information about all workstations, but only about the one that triggers the policy. Second, in this policy the while loop continues until the number of designed tasks reaches the maximum number of tasks that can be generated. This is due to the fact that the number of released and active tasks per workstation is limited by `WS_RELEASED_TASK_CAPACITY`. This design choice is intended to prevent overcrowding near the workstation in scenarios where the number of robots significantly exceeds the system's processing capacity (a large fleet of robots serving a small number of slow workstations, for instance), which would otherwise lead to unnecessary congestion.

8.2 Global optimization model

The model is a static optimization problem split into two stages: the first stage assigns orders to workstations and pods to orders, while the second stage designs tasks and schedules them over a time-space network. The two stages are connected by the decision variables of the first stage, which are used as input parameters for the second stage.

²Vorrei migliorarla cercando di creare dei cluster in base alle sku required dagli order alle workstation

8.2.1 Time-space network

The time-space network is constructed at the beginning of each simulation run, when the `OptManager` class is instantiated. The network is built by considering arcs of the form

$$((s_s, t_s), (s_d, t_d)), \quad t_s < t_d,$$

where each time-space tuple (s, t) represents a node characterized by a spatial position and a temporal coordinate. The spatial position corresponds to the cell identifier within the warehouse grid, while the time coordinate is defined relative to the considered optimization run.

Not all possible arcs are included in the network, as this would significantly increase its size. Specifically, only two classes of arcs are considered:

- *travelling arcs*, namely arcs for which either the source location or the destination location corresponds to a workstation, and such that $t_d - t_s$ is sufficient for the pod to reach the workstation under deterministic travel times;
- *idle arcs*, namely arcs satisfying

$$s_s = s_d, \quad t_d = t_s + 1,$$

where s_s corresponds to a pod storage location.

8.2.2 First Stage: Order-Workstation and Order-Item-Pod Assignments

The first stage assigns orders to workstations and items to pods. The decision variables are:

- $z_{mw} = 1$ if order m is assigned to workstation w , 0 otherwise.
- $x_{imp} = 1$ if item i of order m is retrieved from pod p , 0 otherwise.
- $y_{wp} = 1$ if pod p is assigned to workstation w , 0 otherwise.

In the implementation, x_{imp} is stored with two indices only, since not all items i are required by order m ; a bijective map $(i, m) \rightarrow im$ tracks the correspondence between the two representations.

$$\min \sum_{w \in \mathcal{W}} \sum_{p \in \mathcal{P}} y_{wp} \tag{1}$$

$$\text{s.t.} \quad \sum_{w \in \mathcal{W}} z_{mw} = 1 \quad \forall m \in \mathcal{M} \tag{2}$$

$$\sum_{p \in \mathcal{P}_i} x_{imp} = 1 \quad \forall m \in \mathcal{M}, i \in \mathcal{I}_m \tag{3}$$

$$y_{wp} \geq x_{imp} + z_{mw} - 1 \quad \forall w \in \mathcal{W}, m \in \mathcal{M}, i \in \mathcal{I}_m, p \in \mathcal{P}_i \tag{4}$$

$$\sum_{m \in \mathcal{M}} |\mathcal{I}_m| z_{mw} \leq \bar{I} \quad \forall w \in \mathcal{W} \tag{5}$$

$$\sum_{m \in \mathcal{M}} |\mathcal{I}_m| z_{mw} \geq \underline{I} \quad \forall w \in \mathcal{W} \tag{6}$$

$$z_{mw} = 1 \quad \forall m \in \mathcal{M}_0 \tag{7}$$

where $\mathcal{M}_0 \subset \mathcal{M}$ is the subset of orders whose workstation is already fixed by the current warehouse state. The objective minimises the total number of (workstation, pod) pairs, which is a proxy for the number of pod movements. Constraints (5)–(6) enforce workload balance; the bounds are set to $\bar{I} = 1.15 \frac{\sum_m |\mathcal{I}_m|}{|\mathcal{W}|}$ and $\underline{I} = 0.85 \frac{\sum_m |\mathcal{I}_m|}{|\mathcal{W}|}$.

The problem is solved via the local search described in Section 8.2.4; its output, the assignments z and x , is passed as input to the second stage.

8.2.3 Second Stage: Task Design and Scheduling

Given the assignments from Stage 1, the second stage schedules pod movements over the time-space network. The decision variables are:

- $x_{imt} = 1$ if item i of order m has been picked by time t , 0 otherwise.

- $f_{mt} = 1$ if at least one item of order m has been picked by time $t - 1$, 0 otherwise.
- $g_{mt} = 1$ if order m is completed by time $t - 1$, 0 otherwise.
- $v_{mt} = 1$ if order m is open at time t , 0 otherwise.
- $y_{pa} = 1$ if arc a in the time-space network is selected for pod p , 0 otherwise.

As in Stage 1, x_{imt} is stored with two indices via the same bijective map. The variable y_{pa} is defined only for pods assigned to at least one workstation in Stage 1.

$$\max \quad \sum_{m \in \mathcal{M}} x_{im,T} - \sum_{m \in \mathcal{M}} \sum_{t \in \mathcal{T}} \beta_m(t) (1 - g_{mt}) \quad (8)$$

$$\text{s.t.} \quad \sum_{m \in \mathcal{M}(w)} v_{mt} \leq C_w \quad \forall w \in \mathcal{W}, t \in \mathcal{T} \quad (9)$$

$$\sum_{m \in \mathcal{M}(w)} \sum_{i \in \mathcal{I}_m} \delta^{\text{item}}(x_{imt} - x_{im,t-1}) + \sum_{p \in \mathcal{P}} \sum_{a \in \mathcal{A}_{N(w,t)}^+} \delta^{\text{pod}} y_{pa} \leq \Delta \quad \forall w \in \mathcal{W}, t \in \mathcal{T} \setminus \{0\} \quad (10)$$

$$\sum_{a \in \mathcal{A}_{N_p^0}^+} y_{pa} = 1 \quad \forall p \in \mathcal{P} \quad (11)$$

$$\sum_{a \in \mathcal{A}_n^-} y_{pa} - \sum_{a \in \mathcal{A}_n^+} y_{pa} = 0 \quad \forall p \in \mathcal{P}, n \in \mathcal{N} \setminus (\mathcal{N}_{\text{end}} \cup \{N_p^0\}) \quad (12)$$

$$x_{imt} - x_{im,t-1} \leq \sum_{a \in \mathcal{A}_{N(w,t)}^+} y_{pa} \quad \forall w, m \in \mathcal{M}(w), i \in \mathcal{I}_m, t \in \mathcal{T} \setminus \{0\} \quad (13)$$

$$x_{imt} \geq x_{im,t-1} \quad \forall m, i \in \mathcal{I}_m, t \in \mathcal{T} \setminus \{0\} \quad (14)$$

$$v_{mt} = f_{mt} - g_{mt} \quad \forall m, t \in \mathcal{T} \setminus \{0\} \quad (15)$$

$$f_{mt} \geq x_{imt}, \quad g_{mt} \leq x_{imt} \quad \forall m, i \in \mathcal{I}_m, t \in \mathcal{T} \quad (16)$$

$$g_{m,t+1} \geq \sum_{i \in \mathcal{I}_m} x_{imt} - (|\mathcal{I}_m| - 1) \quad \forall m, t \in \mathcal{T} \quad (17)$$

$$v_{mt} \geq v_{m,t-1} - g_{mt} \quad \forall m, t \in \mathcal{T} \quad (18)$$

$$v_{m,0} = 0 \quad \forall m \in \mathcal{M}_0 \quad (19)$$

$$f_{mt} \geq f_{m,t-1}, \quad g_{mt} \geq g_{m,t-1} \quad \forall m, t \in \mathcal{T} \quad (20)$$

$$\sum_{p \in \mathcal{P}} \sum_{a \in \mathcal{A}_p^{\text{mov}}(t)} y_{pa} \leq |\mathcal{R}| \quad \forall t \in \mathcal{T} \quad (21)$$

The objective (8) maximises throughput and penalises uncompleted orders with a weight $\beta_m(t)$ proportional to the time elapsed since order m arrived, preventing orders from being left indefinitely in the backlog. Constraints (19)–(18) enforce initial conditions for orders already open at workstations, for which f_{mt} is set to 1 regardless of whether any item has been picked. Constraint (21) bounds the number of pods simultaneously out of storage, where

$$\mathcal{A}_p^{\text{mov}}(t) = \{a \in \mathcal{A}_p \mid \ell^+(a) \neq s_p, \quad t^-(a) < t \leq t^+(a)\}$$

and $t^-(a)$, $t^+(a)$ denote the departure and arrival times of arc a . The problem is solved via the local search described in Section 8.2.5.

8.2.4 Local Search Stage 1

The Stage-1 local search minimises $\sum_{w,p} y_{wp}$. At each iteration, a random subset of at most 600 candidate moves is generated from three types:

- **Swap.** The workstation assignments of two free orders are exchanged.
- **Move-to.** A single free order is reassigned to a different workstation.
- **Re-pod.** A single item is reassigned to a different pod stocking its SKU.

After each move, y is recomputed deterministically from x and z . Infeasible candidates are discarded; among the remaining ones, the move yielding the lowest objective is accepted. Ties are accepted to allow plateau traversal. Orders already open at a workstation are never moved. The algorithm terminates after 30 iterations or after 5 consecutive iterations without strict improvement.

8.2.5 Local Search Stage 2

Initial solution. The local search requires a feasible starting point for x_{imt} , built by a dedicated construction heuristic. For each workstation w , orders are partitioned into batches of at most C_w orders. Batches are grown from a seed order by iteratively adding the candidate that maximises pod sharing and minimises new pods introduced; underfull batches are subsequently merged via best-fit on the same score. Pod arrivals are then scheduled tentatively, accounting for pod chaining (the departure location is either the pod’s storage cell or the last workstation it was committed to). A batch is committed only if all its required pods found a feasible slot; uncommitted orders are retried individually.

The construction heuristic is summarised in Algorithm 4.

Algorithm 4 Initial Solution Construction

```

1: for each workstation  $w$  do
2:   Partition orders into batches via greedy pod-sharing score
3:   Merge underfull batches with best-fit
4:   Prepend opened orders as first batch
5:   for each batch  $b$  do
6:     Sort pods by order count (desc) and earliest pick time
7:     for each pod  $p$  in sorted order do
8:        $t^* \leftarrow \text{FINDSLOT}(p, \text{earliest feasible time, travel time})$ 
9:       if  $t^* \neq \text{None}$  then register tentative pick at  $t^*$ 
10:      end if
11:    end for
12:    if all pods of at least one order found a slot then
13:      Commit picks; set  $x_{im, t^*} = 1$  for each item  $i$ , order  $m$ 
14:    else
15:      Add orders of the batch to the retry pool
16:    end if
17:  end for
18:  Retry uncommitted orders individually
19: end for
20: return  $x$ 

```

Neighbourhood and move types. The search perturbs only x_{imt} ; all other variables are derived from x . Up to 150 candidate moves are sampled per iteration, allocated 50%–30%–20% across the three classes:

- **Item shift.** The pick time of one item, or of a small random subset of items, is shifted earlier by a fixed number of time steps; the shift magnitude decreases when no improvement has been found in the previous iteration.
- **Order shift.** All items of an order are shifted earlier by a fixed number of time steps.
- **Order swap.** Two orders at the same workstation exchange their position in the picking sequence.

Two-phase feasibility check. To avoid rebuilding y at every candidate evaluation, feasibility is verified in two stages. First, a cheap x -only check verifies the constraints that do not involve pod routing (monotonicity of x , capacity (9), consistency of f, g, v), which are updated incrementally by recomputing only the rows of affected orders. Only for the best x -feasible candidate per iteration are the affected pod rows of y rebuilt and the full constraint check (11)–(21) performed. If this check fails, the second-best candidate is tried before discarding the iteration. The algorithm terminates after 45 iterations or after 3 consecutive iterations without improvement.

References

- [BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. “Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search”. In: *ArXiv* abs/2408.16890 (2024). URL: <https://api.semanticscholar.org/CorpusID:272310402>.